

ROBOT FRAMEWORK AUTOMATION TESTING PROJECT

PROJECT REPORT

Submitted By

AKASH SAHA

Department Name

CSE AIML

Institution Name

INSTITUTE OF ENGINEERING AND MANAGEMENT

Academic Year

2023–2027

Project Area

Test Automation • Robot Framework • Web Automation • Data-Driven Testing

1. Abstract

This project presents a structured test automation framework using Robot Framework. The project focuses on web automation, reusable test keywords, variables, data-driven execution, assertions, test setup and teardown, test tagging, execution control, logging, and automated test reporting. Robot Framework provides a readable keyword-driven syntax, making the test cases easier to understand and maintain. The framework can be extended with browser automation libraries and Python-based custom libraries to support application-specific testing requirements.

2. Introduction

Software testing is an essential part of software development because it verifies whether an application behaves according to its expected requirements. Manual execution of repetitive test cases requires significant time and may introduce inconsistency. Automation testing allows predefined test cases to be executed repeatedly with less manual intervention. Robot Framework provides a keyword-driven approach in which test cases can be expressed in a human-readable format while still supporting advanced automation through libraries and Python extensions.

3. Problem Statement

Traditional manual testing becomes difficult when the same browser interactions, form submissions, validations, and test scenarios have to be executed repeatedly. Maintaining large numbers of test cases can also become challenging when test data changes. The project addresses these challenges by using reusable Robot Framework keywords, variables, data-driven test execution, assertions, lifecycle management, tags, logs, and automated reports.

4. Objectives

The main objectives are to develop readable automated test cases using Robot Framework; automate browser-based application workflows; use variables for reusable test data; implement data-driven testing using external data; create custom keywords using Python; perform assertions and verification; configure common test setup and teardown; organize tests using tags; support controlled test execution; generate execution logs and HTML reports; and create a maintainable automation structure.

5. Scope

The project covers the core capabilities represented in the provided task specification: basic Robot Framework syntax and keywords, browser navigation, form input, element verification, variables, external test data, custom Python keywords, BuiltIn library operations, assertions, API response verification, test setup and teardown, tagging, selective execution, parallel execution concepts, HTML reports, and custom logging.

6. Technology Overview

Robot Framework: Keyword-driven automation framework used to define and execute test cases.

Browser Automation Library: Used to interact with web pages and browser elements in browser-based scenarios.

Python: Used for custom Robot Framework libraries and keywords when built-in or existing library functionality is insufficient.

BuiltIn Library: Provides commonly used keywords for variables, strings, assertions, control flow, and other test operations.

External Test Data: Allows the same test logic to be executed using multiple input datasets.

HTML Reporting: Robot Framework produces execution reports and logs containing test results and execution details.

7. Project Structure

```
Robot_Framework_Project/  
■  
■ ■ ■ ■ tests/  
■ ■ ■ ■ login_tests.robot  
■ ■ ■ ■ data_driven_tests.robot  
■ ■ ■ ■ api_tests.robot  
■  
■ ■ ■ ■ resources/  
■ ■ ■ ■ common_keywords.robot  
■ ■ ■ ■ variables.robot  
■  
■ ■ ■ ■ libraries/  
■ ■ ■ ■ custom_library.py  
■  
■ ■ ■ ■ test_data/  
■ ■ ■ ■ test_data.csv  
■  
■ ■ ■ ■ results/  
■ ■ ■ ■ report.html  
■ ■ ■ ■ log.html
```

The structure above represents a recommended organization for the capabilities described in the provided task specification. The uploaded source specifies the required functionality but does not provide a concrete project directory structure; therefore, the structure is presented as an organized implementation model rather than as a claim about an existing folder layout.

8. Basic Syntax and Keywords

Robot Framework test cases are written using tables such as Settings, Variables, Test Cases, and Keywords. Keywords represent reusable actions. Browser automation scenarios can use keywords for opening a browser, navigating to a URL, entering text into fields, clicking elements, and validating page content. The provided task specification specifically includes using **Input Text** to enter form data and **Page Should Contain Element** to verify the presence of an element.

9. Variables and Data-Driven Automation

Variables allow test data such as usernames and passwords to be stored separately from the automation steps. This improves reuse and makes test maintenance easier. Data-driven automation extends this concept by executing the same set of actions against multiple datasets stored externally. The provided specification requires a test that reads test data from an external file and performs the same actions for multiple data records.

10. Custom Keywords and Python Libraries

Robot Framework allows reusable custom keywords to be created. Python can be used to implement application-specific functionality that is not directly available through existing Robot Framework libraries. For example, a Python library can provide a keyword for calculating the sum of two numbers. The Robot Framework **BuiltIn** library can also be used for common string manipulation and mathematical or validation operations.

11. Assertions and Verification

Assertions verify that the actual result of a test matches the expected result. Web tests can validate that a required element is present or that displayed information matches expectations. For API testing, the provided specification specifically identifies the **Should Be Equal As Strings** keyword for comparing an expected response with the actual response. A failed assertion causes the relevant test case to fail and is recorded in the execution results.

12. Test Setup and Teardown

Common test setup can be configured so that required actions are automatically performed before each test case. The provided specification gives browser opening and login as an example of common setup. Teardown performs cleanup after a test case, such as logging out or releasing resources. Centralizing these operations avoids repeating the same setup and cleanup steps throughout individual test cases.

13. Tags and Test Execution

Tags provide a mechanism for categorizing test cases and suites. They can be used to select a particular group of tests during execution. The provided task specification identifies the **--include** and **--exclude** command-line options for controlling which tagged tests are executed. Tagging is useful for separating smoke, regression, functional, API, or other logical test groups.

14. Parallel Test Execution

Parallel execution allows independent tests to run concurrently and can reduce total execution time. The provided specification identifies the **--processes** option as the mechanism to utilize for parallel test execution. Parallel execution should be applied only when tests are sufficiently independent and the environment and test data can safely support concurrent operations.

15. Reports and Logs

Robot Framework provides HTML-based execution reporting. The generated report contains information about test cases, their outcomes, and execution details. The **Log** keyword can be used to add custom messages during execution. Logging is particularly useful for understanding test flow and diagnosing failures during automation runs.

16. Sample Robot Framework Test Case

```
*** Settings ***
Library SeleniumLibrary

*** Test Cases ***
Open Website And Verify Element
    Open Browser https://example.com chrome
    Page Should Contain Element xpath=//body
    Log Website opened and element verified
    Close Browser
```

The exact URL and locator should be replaced with the application under test. The example demonstrates the structure of a Robot Framework test case and the use of browser actions, verification, logging, and cleanup.

17. Data-Driven Test Concept

A data-driven test separates test logic from test values. For example, a login test can use several username and password combinations while keeping the browser interaction steps unchanged. The external dataset can contain multiple records, and the same test workflow can be executed for each record. This approach reduces duplicate test-case code and improves coverage.

18. Custom Python Keyword Concept

A custom Python library can expose application-specific functions as Robot Framework keywords. For example, a Python method that accepts two numbers and returns their sum can be exposed as a keyword and called from a Robot Framework test. This mechanism allows complex logic or reusable utilities to remain in Python while test scenarios remain readable in Robot Framework.

19. Test Execution Workflow

The general workflow is: define the test requirement; create the Robot Framework test case; declare variables and libraries; configure setup and teardown; execute browser or API actions; perform assertions; add useful log messages; apply tags where required; execute selected or complete test suites; and review the generated report and log files.

20. Execution Commands

Execute a Robot Framework suite:
`robot tests/`

Execute tests with a selected tag:
`robot --include smoke tests/`

Exclude a tag:
`robot --exclude smoke tests/`

Generate results in a specific directory:
`robot --outputdir results tests/`

21. Advantages

The framework provides readable test cases, reusable keywords, separation of test data from test logic, support for custom Python functionality, built-in verification mechanisms, reusable setup and teardown, test categorization through tags, execution logging, and HTML reporting. These capabilities make the automation suite easier to understand, execute, and maintain.

22. Limitations

Browser automation depends on a stable test environment and compatible browser automation libraries. External test data must be maintained correctly. Parallel execution requires tests to be sufficiently independent. Custom Python libraries also require appropriate dependency and error management. The exact implementation of API, browser, and data-driven functionality depends on the application under test and the libraries selected.

23. Future Enhancements

The framework can be extended with Page Object or Screen Object abstractions, centralized configuration, reusable resource files, richer data sources, API and UI integration, screenshots on failure, CI/CD integration, cross-browser execution, parallel test execution, database validation, enhanced reporting, and containerized execution environments.

24. Conclusion

The project demonstrates the use of Robot Framework for structured and maintainable test automation. The covered capabilities include basic syntax and keywords, web interaction, variables, data-driven testing, custom Python keywords, BuiltIn operations, assertions, setup and teardown, tags, execution control, reports, and logging. Together, these features provide a strong foundation for automating functional test scenarios while keeping test cases readable and reusable.

25. References

1. Robot Framework documentation and user guide.
2. Selenium/browser automation library documentation used with Robot Framework.
3. Python documentation for custom library development.
4. The project task specification provided for this report.